

Manipal University

Name: Devanshi Rana

Email: rana.2502051097@mu.j.manipal.edu

Roll no: 2502051097

Phone: 7600927404

Branch: Manipal

Department: CSE - T1 - Batch 2

Batch: 2029

Degree: B.Tech

Scan to verify results



NeoColab_Manipal_2029_DSA Lab Course

Neocolab_Manipal_DSA_Week 9_PAH

Attempt : 2

Total Mark : 20

Marks Obtained : 20

Section 1 : coding

1. Problem Statement

Rosh is managing a smart circular queue-based parking system with exactly 10 parking spots, indexed from 0 to 9. When a car arrives, she parks it in the next available spot. If a car's number is entered and the lot is full, she should be notified without consuming the car number. Cars leave the parking lot in the same order they arrived. Rosh also needs to handle cases when the lot is empty, and when an invalid operation code is entered.

Write a program to simulate this parking system with support for inserting and retrieving car numbers, full/empty conditions, and input validation.

Input Format

The first line contains an integer choice representing the operation type.

If choice == 1: Next line contains an integer carNumber to be parked.

If choice == 2: Retrieve the car at the front.

If choice == 3: Exit the program.

Any other value: Treated as an invalid choice.

Repeat until choice 3 is entered.

Output Format

For Insert operation (choice 1):

If parking is full, print "Parking lot is full."

Else: print "Car <carNumber> parked in spot <spotIndex>."

For Retrieve operation (choice 2):

If parking is empty: print "Parking lot is empty."

Else:

Print "Car <carNumber> retrieved from spot <spotIndex>."

For Exit operation (choice 3):

Program terminates without any output.

For any Invalid choice (not 1, 2, or 3): print "Invalid choice."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

123

1

456

2

3

Output: Car 123 parked in spot 0.

Car 456 parked in spot 1.

Car 123 retrieved from spot 0.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define SIZE 10
```

```
int queue[SIZE];
```

```
int front = -1, rear = -1;
```

```
int isFull() {
```

```
    return ((rear + 1) % SIZE == front);
```

```
}
```

```
int isEmpty() {
```

```
    return (front == -1);
```

```
}
```

```
void parkCar(int carNumber) {
```

```
    if (isFull()) {
```

```
        printf("Parking lot is full.\n");
```

```
        return;
```

```
}
```

```
    if (isEmpty()) {
```

```
        front = rear = 0;
```

```
    } else {
```

```
        rear = (rear + 1) % SIZE;
```

```
    }
```

```
    queue[rear] = carNumber;
```

```
    printf("Car %d parked in spot %d.\n", carNumber, rear);
```

```
}
```

```
void retrieveCar() {
```

```

if (isEmpty()) {
    printf("Parking lot is empty.\n");
    return;
}
int carNumber = queue[front];
int spotIndex = front;
printf("Car %d retrieved from spot %d.\n", carNumber, spotIndex);
if (front == rear) {
    front = rear = -1;
} else {
    front = (front + 1) % SIZE;
}
}
int main() {
    int choice, carNumber;
    while (1) {
        if (scanf("%d", &choice) != 1) break;
        switch (choice) {
            case 1:
                if (isFull()) {
                    printf("Parking lot is full.\n");
                } else {
                    scanf("%d", &carNumber);
                    parkCar(carNumber);
                }
                break;
            case 2:
                retrieveCar();
                break;
            case 3:
                return 0; // Exit without printing anything
            default:
                printf("Invalid choice.\n");
        }
    }
    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Design and implement an airport runway scheduling system using a circular queue data structure. The system should manage the planes waiting to land or take off and control their access to the runway for efficient air traffic flow.

The system should allow the user to perform the following operations:

Enqueue a plane into the queue with a unique plane ID and plane type (landing or taking). Dequeue a plane from the queue (when it has access to the runway). Display the current state of the queue with the plane IDs and types. Exit the program.

The program should be able to handle invalid inputs and display appropriate error messages.

The program should also be able to handle the circular nature of the queue, ensuring that when the last element is reached, the next enqueued element is added at the beginning of the queue.

Note:

There is a new line space after the last line of the output. The queue can hold only 9 elements.

Input Format

Each line starting with 1 is followed by two values:

- A number (flight number or ID).
- A string (operation type: "Landing" or "Taking"). This operation inserts the flight ID and operation type into the queue.

Each line starting with 2 indicates that the flight at the front of the queue should be removed.

Each line starting with 3 should output the current state of the queue.

Output Format

The output displays the message if the choice is 3, also if the queue is not empty, in the following format:

"Queue elements are:

<flight_id>, <operation_type>,

<flight_id>, <operation_type>,

..."

For choice 3 if the queue is empty, it prints "Queue is empty".

For choice 1, if the queue is full, it prints "Queue is full", the queue can hold only 9 elements.

The output prints "Invalid choice" if the input choice is not 1, 2, 3 or 4.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

101

Landing

1

102

Taking

3

2

3

2

3

4

Output: Queue elements are:

101, Landing

102, Taking

Queue elements are:

102, Taking

Queue is empty

Answer

```
#include <stdio.h>
```

```
#include <string.h>
#include <stdlib.h>
```

```
#define MAX 9
```

```
typedef struct {
    int id;
    char type[20];
} Plane;
```

```
typedef struct {
    Plane queue[MAX];
    int front, rear, count;
} CircularQueue;
```

```
void initializeQueue(CircularQueue *q) {
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}
```

```
int isFull(CircularQueue *q) {
    return q->count == MAX;
}
```

```
int isEmpty(CircularQueue *q) {
    return q->count == 0;
}
```

```
void enqueue(CircularQueue *q, int id, char *type) {
    if (isFull(q)) {
        printf("Queue is full\n");
        return;
    }
    q->rear = (q->rear + 1) % MAX;
    q->queue[q->rear].id = id;
    strcpy(q->queue[q->rear].type, type);
    q->count++;
}
```

```
void dequeue(CircularQueue *q) {
    if (isEmpty(q)) {
```

```
        return;
    }
    q->front = (q->front + 1) % MAX;
    q->count--;
}
```

```
void display(CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }
```

```
    printf("Queue elements are:\n");
```

```
    for (int i = 0; i < q->count; i++) {
        int index = (q->front + i) % MAX;
        printf("%d, %s\n", q->queue[index].id, q->queue[index].type);
    }
}
```

```
int main() {
    CircularQueue q;
    initializeQueue(&q);
```

```
    int choice, id;
    char type[20];
```

```
    while (1) {
        if (scanf("%d", &choice) != 1)
            break;
```

```
        switch (choice) {
            case 1:
                if (scanf("%d", &id) != 1) break;
                scanf("%s", type);
                enqueue(&q, id, type);
                break;
```

```
            case 2:
                dequeue(&q);
                break;
```



```
        case 3:
            display(&q);
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
            break;
    }
}

return 0;
```

Status : Correct

Marks : 10/10